

Example:

Ship Lines Optimization in Southeastern Asia

In this file, we will demonstrate how to optimize the shipline network step by step using the `cma` package.

We focus on a specific region: the maritime network within Southeast Asia. This region includes the ports of ten ASEAN countries and some ports in southern China, basically covers the main ports along the coast of the South China Sea.

First of all, please import the `cma` package.

```
import cma
```

This region fully encompasses the following 10 countries (as well as southern China, Taiwan, and parts of South Asia):

```
southeastern_asia = [  
    'Vietnam', 'Myanmar', 'Laos', 'Thailand', 'Cambodia',  
    'Indonesia', 'Philippines', 'Malaysia', 'Singapore',  
    'Brunei', 'Timor-Leste'  
]
```

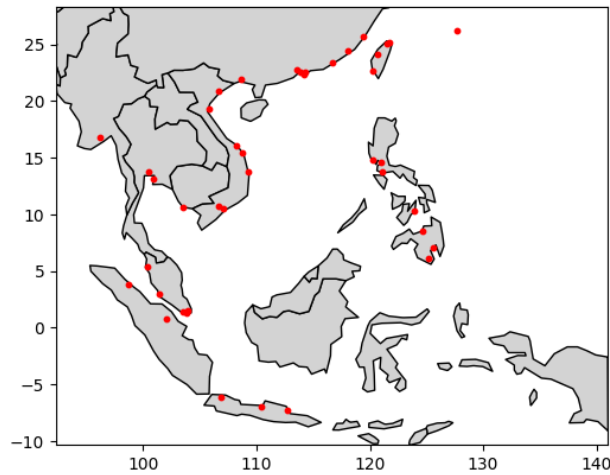
1. Ports and Demands

Southeast Asia, as a key region for global maritime trade, is home to many ports.

Let's display the distribution of all the ports in this region.

```
# load vessels data  
vesselpool = cma.read_vessel_class_data()  
  
# load ports data  
_, portpool = cma.read_port_data()  
  
# filtered by `southeastern_asia` region  
_, ax, portpool_2 = portpool.plot(southeastern_asia)
```

In Total 41 ports are plotted



From the image above, we have a visualization of the distribution of ports in the Southeast Asia region: There are 41 ports considered, as the program tells us.

Next, let's examine the freight demand among these ports. In fact, not all port pairs have freight demand between them.

(Please note that in this example, we are focusing solely on the demand within the region and excluding the demand between the region and external areas.)

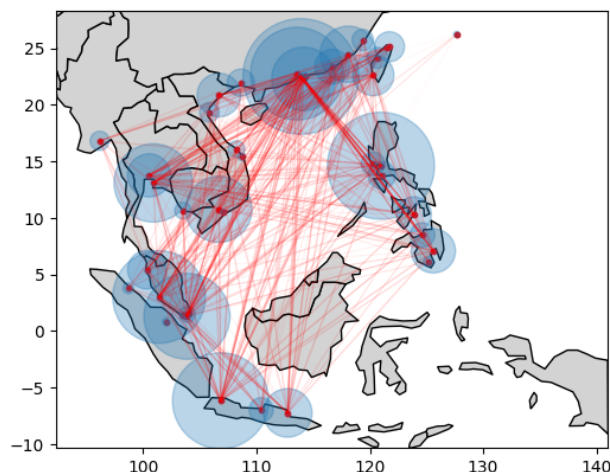
```
# load weekly demand data
_, weekly_demand = cma.read_demand_data(portpool_2)

# load mutual sailing distance data
dist_mat = cma.read_sailing_distance_data(portpool_2)

# create a new pool of ports with richer information
portgraph = cma.PortGraph(portpool_2, dist_mat, weekly_demand)
num_ports = portgraph.get_number_of_ports()
print(f'In total {num_ports} have freight demand')

# plot the demands between ports
_, ax, _ = portgraph.plot(southeastern_asia)
```

In total 40 have freight demand



In the above image, there are a total of 40 ports involved in cargo transportation, which is only slightly different from the total number of ports.

The traffic level at each port (total inbound and outbound cargo) is represented by light blue circles. The larger the circle, the greater the volume of cargo handled at the port.

The red lines represent the demand between ports, with the color intensity and line thickness indicating the volume of cargo.

All data related to demand has been inherited into the `PortGraph` class.

Besides the demand data, this class also contains the nautical distance between any two ports. We have defined methods such as `get_distance` and `get_demand` in the class.

Note:

It is worth noting that the nautical distance is not available for every pair of ports. There are missing data.

For those edges with missing distance, we currently set infinity as a placeholder to indicate that the sea route is unavailable.

There are 126 such missing values in `dist_mat` in total. If these edges appear in a route, it means the route is not feasible.

```
# statistics
pairs = portgraph.get_pairs_missing_distance()
len(pairs)
```

126

2. Network Optimization

Now, let's move on to the maritime route optimization section.

First, we will read the existing routes from the data. It is important to emphasize that Our current model only considers the most standard routes:

1. Ships cannot visit the same port consecutively, and
2. The same directed segment (of two ports) cannot appear twice in the route

The second restriction is mainly to facilitate path searching in the cargo allocation problem and decrease the size of action space of line adjustment.

```
# load current lines
current_lines, _ = cma.read_current_line_data(portgraph)
print(f'Successfully load {len(current_lines)} lines')
```

Warning: Invalid since a port is visited consecutively.
Warning: Invalid since a port is visited consecutively.
Warning: Invalid since a slot is repeated.
Warning: Invalid since a port is visited consecutively.
Warning: Invalid since a port is visited consecutively.
Successfully load 10 lines

```
# create a graph network
servicegraph = cma.ServiceGraph(current_lines)
servicegraph
```

```
CP2CNC -- [CNSHK, HKHKG, CNNSA, PHMNL, PHBTG, PHDVO]
CP3CNC -- [TWKHH, CNSHK, HKHKG, PHSFS, PHCEB, PHCGY]
HHX3CNC -- [CNNSA, CNSHK, HKHKG, VNHPH]
JSSCNC -- [SGSIN, MYPKG, IDJKT, IDSUB]
LCXPCNC -- [THLCH, SGSIN]
RBHCNC -- [THBKK, THLCH, VNHPH, CNNSA, CNSHK, THLCH]
RMNCNC -- [PHSFS, PHMNL, SGSIN]
SGSCNC -- [SGSIN, VNNGN]
SPXCNC -- [SGSIN, PHDVO, PHGES]
TIXCNC -- [THLCH, IDJKT]
```

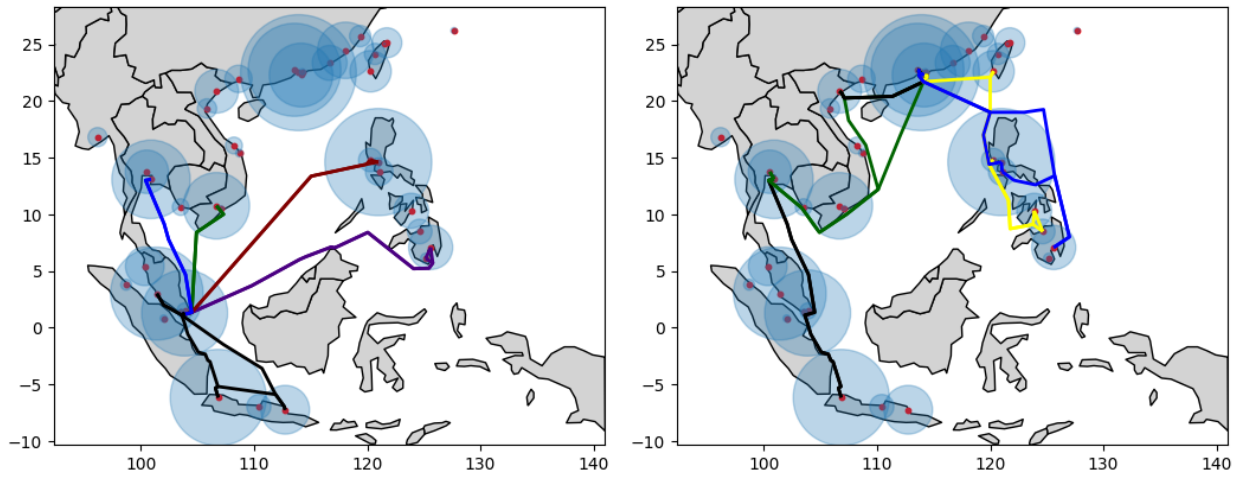
2.1. A Glance of Current Lines

Alright, we have successfully read the 10 existing maritime routes.

Their paths are plotted as follows:

```
servicegraph.plot( \
    lines_info=[
        ('SPXCNC', 'indigo'), ('SGSCNC', 'darkgreen'),
        ('RMNCNC', 'maroon'), ('LCXPCNC', 'blue'),
        ('JSSCNC', 'black'),
    ], selected_countries=southeastern_asia,
    portgraph=portgraph
)

servicegraph.plot( \
    lines_info=[
        ('TIXCNC', 'black'), ('RBHCNC', 'darkgreen'),
        ('HHX3CNC', 'black'), ('CP3CNC', 'yellow'),
        ('CP2CNC', 'blue'),
    ], selected_countries=southeastern_asia,
    portgraph=portgraph
)
```

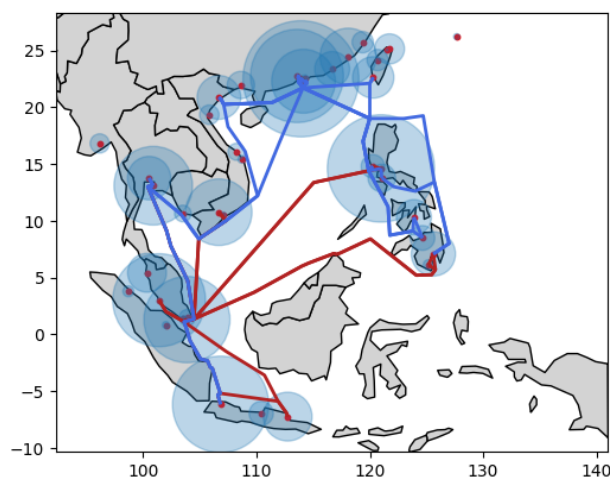


To make it easier to distinguish, I have divided the 10 routes into two groups and plotted them in two separate images.

The first image depicts a web-like network centered around Singapore, while the second image shows a crescent-shaped ring encircling the South China Sea.

By combining the two route maps, we obtain a complete shipping route map as shown below.

```
servicegraph.plot( \
    lines_info=[
        ('SPXCNC', 'firebrick'), ('SGSCNC', 'firebrick'),
        ('RMNCNC', 'firebrick'), ('LCXPCNC', 'firebrick'),
        ('JSSCNC', 'firebrick'),
        ('TIXCNC', 'royalblue'), ('RBHCNC', 'royalblue'),
        ('HHX3CNC', 'royalblue'), ('CP3CNC', 'royalblue'),
        ('CP2CNC', 'royalblue'),
    ], selected_countries=southeastern_asia,
    portgraph=portgraph
)
```



From the above image, we can gather at least two pieces of information:

- First, not all ports are covered by this shipping network;

- Second, not all demands are being fulfilled.

Through a simple statistics, we found that 21 ports are not connected to the current network, which is more than half of the total.

Among all the demand orders, 127 can be fulfilled within two transshipments, while 246 cannot be met by this requirement. These unfulfilled cargo transport demands account for around one-third of the total.

```
# statistics
isolated_ports = servicegraph.get_isolated_ports(portgraph)
print('Number of isolated ports =', len(isolated_ports))
```

Number of isolated ports = 21

```
# statistics
pathinfo = servicegraph.get_all_paths(portgraph)
print('Number of fulfilled demand order  =', len(pathinfo['od_pairs']))
print('Number of unfulfilled demand order =', len(pathinfo['unconnected']))
print('Sum of unfulfilled demand =', sum(pathinfo['unconnected_demand']))
print('Sum of fulfilled demand  =', sum(pathinfo['od_pairs_demand']))
```

Number of fulfilled demand order = 127

Number of unfulfilled demand order = 246

Sum of unfulfilled demand = 9446.0

Sum of fulfilled demand = 17971.0

Next, let's estimate the shipping cost of the current network.

First, due to the large number of unmet demands, **for simplicity**, we will assign a penalty cost of 1000 to each unmet demand.

Note:

The magnitude of this penalty will significantly affect the behavior of our algorithm.

It is a tune parameter, setted in the argument of function `solve_approximated_cost` in [cma.servicegraph.py](https://github.com/maoyan/cma.servicegraph.py).

If you set this parameter sufficiently large, it basically means you don't allow any demand unfulfilled.

```
# create a week prediction model
week_predictor = cma.create_week_predictor()

# solve the optimal demand fulfillment
servicegraph.solve_approximated_cost(portgraph, vesselpool, week_predictor)
servicegraph.total_cost()
```

11984527.287209721

2.2. MCTS Algorithm

Now, let's run the Monte Carlo Tree Search (MCTS) algorithm to optimize the current routes.

First, we create a search tree with the current routes as the root node.

```
tree = cma.MonteCarloTree(servicegraph=servicegraph,
    portgraph=portgraph, vesselpool=vesselpool,
    discount_fac=0.5, # control the simulation depth
    c_param=0.5,      # balance exploration and exploitation
    max_depth=20,     # how many steps of adjustments are allowed
    week_predict_model=week_predictor)
```

Here, we need to explain two key parameters:

- `discount_fac`: greater than 0 and less than 1.

This parameter controls the simulation depth of future adjustments. The larger the discount factor, the more simulations will be taken, and the algorithm is slower.

In deep learning (e.g., AlphaGo), the discount factor is set to 1 because the algorithm can afford to prioritize long-term gains.

However, in our case, the discount factor needs to be less than 1, as the adjustments to the routes must be completed within a limited number of steps. This ensures that the algorithm focuses more on short-term rewards and optimization, as the route adjustments need to be made efficiently and within a constrained timeframe.

- `c_param`: greater than 0.

This parameter balance the exploration and exploitation. The larger the value of `c`, the more the program favors exploration, meaning it tends to jump between different branches. Conversely, the smaller the value of `c`, the more the program favors exploitation, meaning it tends to delve deeper into the current branch.

Users can test for different `c` and check the behavior changes of this program. It should be fun.

Next, let's run the searching algorithm for 5 minutes:

```
import time
from IPython.display import clear_output

run_time_seconds = 5 * 60 # 5 minutes
start_time = time.time()

while True:
    if time.time() - start_time >= run_time_seconds:
        print('Run time finished')
        break
    tree.run(10, True)
    clear_output(wait=False)
    print(tree.recorder())
    print('-----')
    tree.display_best_node(portgraph)
```

```
{'num_expand': 145, 'num_rollout': 105}
```

Best Action Trace:

Action 1:

For the #5 line "RBHCNC -- [THBKK, THLCH, VNHPH, CNNSA, CNSHK, THLCH]",
Add Port "IDJKT" in the segment (THBKK, THLCH).

Siblings = 2

Original cost = 11984527.287209721

Updated cost = 10596351.832742043

Action 2:

For the #1 line "CP3CNC -- [TWKHH, CNSHK, HKHKG, PHSFS, PHCEB, PHCGY]",
Add Port "IDSRG" in the segment (PHCEB, PHCGY).

Siblings = 3

Original cost = 10596351.832742043

Updated cost = 10513570.015357453

Action 3:

For the #0 line "CP2CNC -- [CNSHK, HKHKG, CNNSA, PHMNL, PHBTG, PHDVO]",
Add Port "TWTXG" in the segment (CNNSA, PHMNL).

Siblings = 5

Original cost = 10513570.015357453

Updated cost = 10396746.5978427

Action 4:

For the #6 line "RMNCNC -- [PHSFS, PHMNL, SGSIN]",
Add Port "HKHKG" in the segment (PHSFS, PHMNL).

Siblings = 9

Original cost = 10396746.5978427

Updated cost = 7117950.52241344

Action 5:

For the #8 line "SPXCNC -- [SGSIN, PHDVO, PHGES]",
Add Port "CNXMN" in the segment (PHDVO, PHGES).

Siblings = 1

Original cost = 7117950.52241344

Updated cost = 6137783.852422209

Run time finished

After 5 minutes of searching, the program suggests the above actions.

Note:

Due to the short search time, the expansion range of the search tree is limited, so the obtained result is certainly not the best. User may try extending the search time.

Additionally, the program has a certain degree of randomness, so you may not get similar results each time you run it. Please try multiple times to obtain a more satisfactory result.

As you can see, the program produces a sequence of adjustment actions. After each step, we can calculate the total cost after the network adjustment.

Some results may not appear ideal. Although they have lower costs, they make the overall network more complex. Therefore, users can choose to stop at the step where the cost reduction is most significant.

Users can obtain these adjusted results by `best_node_trace` function:

```
results_trace = tree.best_node_trace()
```

After this simple adjustment, the total cost decreases significantly by around 40%:

```
old_cost = round(servicegraph.total_cost())
new_cost = round(results_trace[-1].graph.total_cost())
print(f'Original graph total cost = {old_cost}')
print(f'Updated graph total cost = {new_cost}')
```

Original graph total cost = 11984527

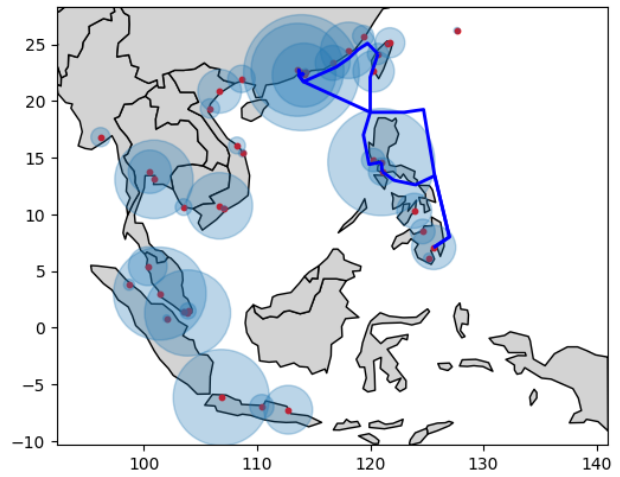
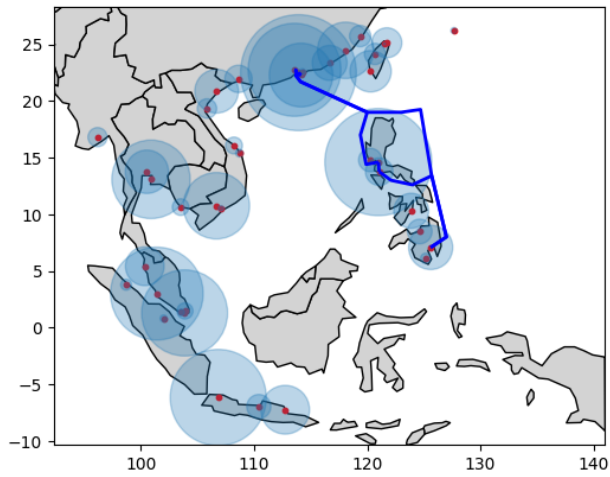
Updated graph total cost = 6137784

2.3. Visualization

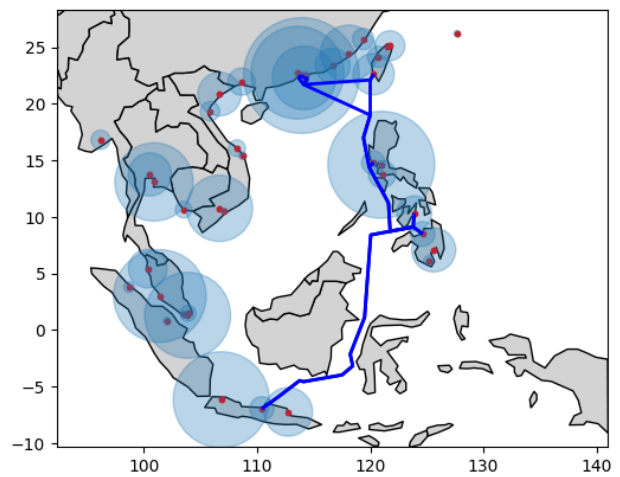
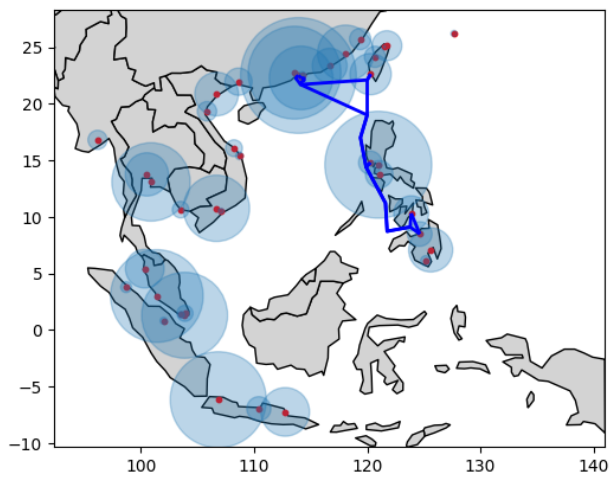
Finally, let's visualize these changes:

Based on the above suggestions, the following 5 lines are adjusted: `CP2CNC`, `CP3CNC`, `RBHCNC`, `RMNCNC` and `SPXCNC`.

```
# CP2CNC: original line
servicegraph.plot( \
    lines_info=[
        ('CP2CNC', 'blue'),
    ], selected_countries=southeastern_asia, portgraph=portgraph,
)
# CP2CNC: updated line
results_trace[-1].graph.plot( \
    lines_info=[
        ('CP2CNC', 'blue'),
    ], selected_countries=southeastern_asia, portgraph=portgraph,
)
```



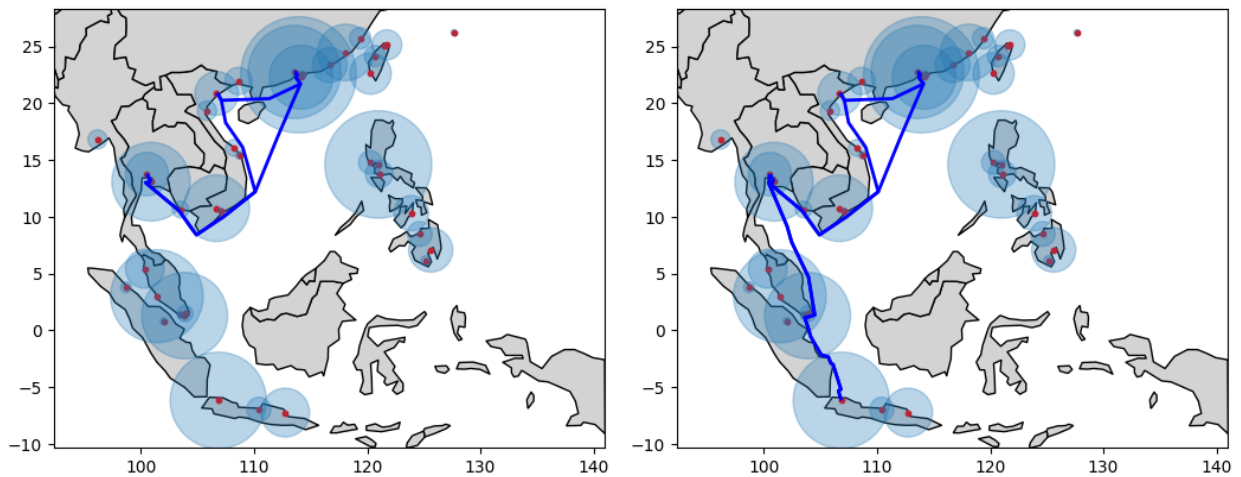
```
# CP3CNC: original line
servicegraph.plot( \
  lines_info=[
    ('CP3CNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
# CP3CNC: updated line
results_trace[-1].graph.plot( \
  lines_info=[
    ('CP3CNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
```



```

# RBHCNC: original line
servicegraph.plot( \
  lines_info=[
    ('RBHCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
# RBHCNC: updated line
results_trace[-1].graph.plot( \
  lines_info=[
    ('RBHCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)

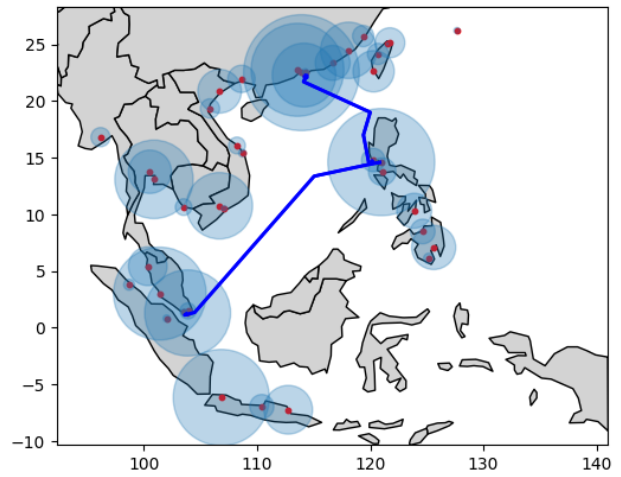
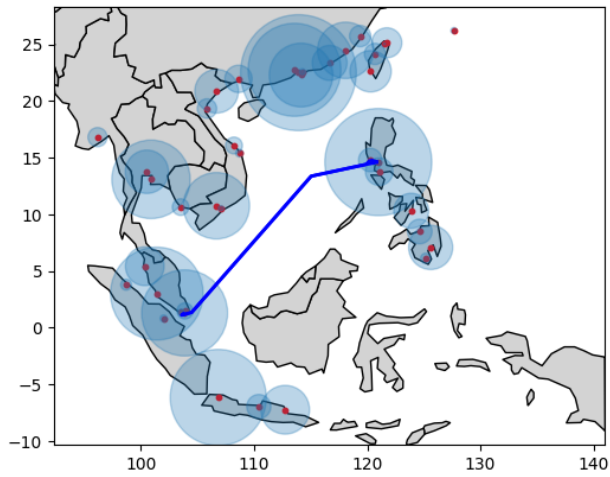
```



```

# RMNCNC: original line
servicegraph.plot( \
  lines_info=[
    ('RMNCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
# RMNCNC: updated line
results_trace[-1].graph.plot( \
  lines_info=[
    ('RMNCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)

```



```
# SPXCNC: original line
servicegraph.plot( \
  lines_info=[
    ('SPXCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
# SPXCNC: updated line
results_trace[-1].graph.plot( \
  lines_info=[
    ('SPXCNC', 'blue'),
  ], selected_countries=southeastern_asia, portgraph=portgraph,
)
```

